

VITyarthi Flipped Course

PROJECT REPORT

Student Performance & Attendance Analyzer

A Python-based command-line application for student data management, academic analysis, attendance tracking, and reporting.

Course	Python Programming
Project Type	Command-Line Application
Data Storage	JSON
Testing	pytest
Version Control	Git / GitHub
Student Name	_____
Registration No.	_____
Institute	VIT Bhopal University

Academic Project Report

1. Introduction

Student Performance & Attendance Analyzer is a modular Python command-line application developed for the Python Programming course. The system provides a structured way to store student information, enter academic marks, record attendance, calculate grades and percentages, analyze overall performance, and generate reports.

The application uses JSON as a lightweight persistent storage mechanism. The project demonstrates core Python programming concepts including functions, conditional logic, loops, exception handling, file handling, modular programming, data validation, and automated testing.

Project Goal: To create a simple, maintainable and testable academic data management system that demonstrates practical application of Python programming concepts.

Project Scope

- Manage basic student records.
- Enter and view marks for multiple subjects.
- Calculate total marks, percentage, and grade.
- Record and calculate attendance percentage.
- Analyze academic performance together with attendance.
- Generate a consolidated student performance report.
- Validate inputs and handle common invalid entries safely.
- Run automated tests using pytest.

2. Problem Statement

Managing student academic information manually can be time-consuming and can make it difficult to keep marks and attendance organized in one place. A small academic management tool can reduce repetitive calculations and provide a consistent way to view student progress.

The proposed solution is a command-line Python application that stores student data in JSON, performs calculations automatically, validates user input, and presents the results through simple menus and reports.

Target Users

- Students who want to organize their academic information.
- Teachers or faculty members who need a simple record and reporting utility.
- Academic administrators for basic student-level analysis.

3. Functional Requirements

The project provides five major functional modules, supported by validation and testing modules.

Module	Main Functions	Input / Output
Student Management	Add, view, search, delete students	Student details → stored/retrieved records
Marks Management	Enter/view marks, total, percentage, grade	Subject marks → academic result
Attendance Management	Enter/view classes and attendance	Class counts → attendance percentage
Performance Analysis	Combine marks and attendance	Academic + attendance data → performance category

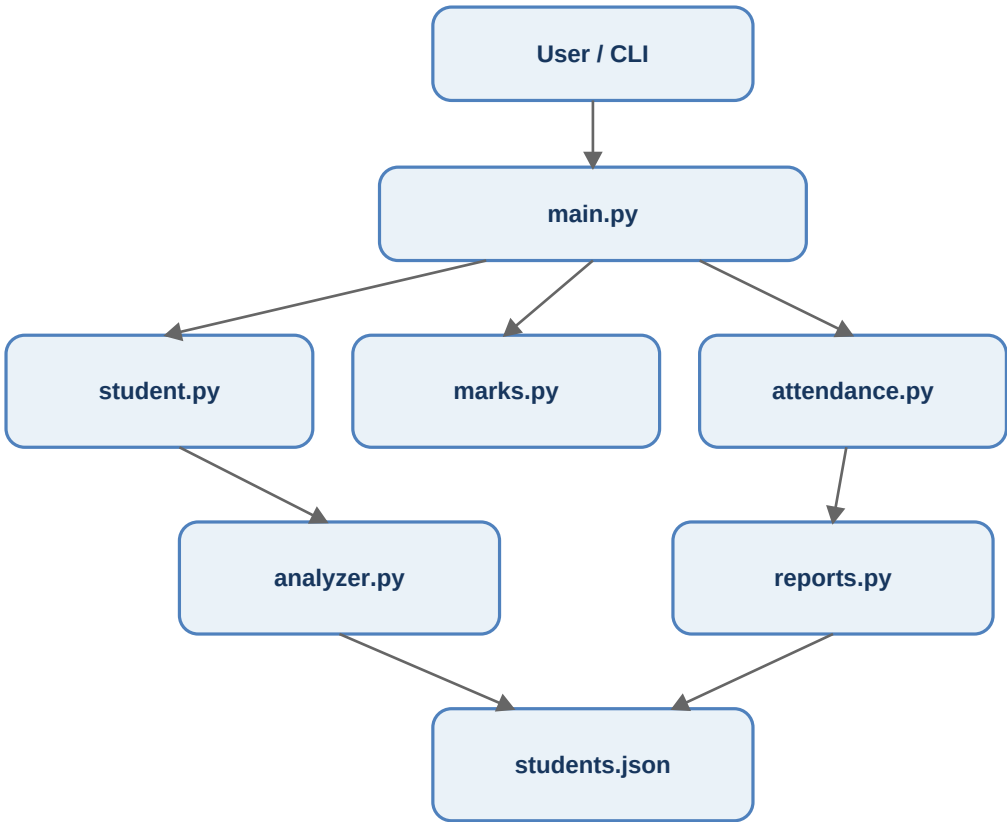
Module	Main Functions	Input / Output
Reports	Generate consolidated report	Stored records → readable report

Input/Output examples: Student ID, name and course are inputs for student management; marks produce total, percentage and grade; class counts produce attendance percentage; analysis produces an overall category.

4. Non-functional Requirements

Requirement	Project Implementation
Usability	Menu-driven command-line interface with clear prompts and messages.
Reliability	Persistent JSON storage and controlled handling of missing records.
Maintainability	Separate Python files for major responsibilities.
Error Handling	Invalid numeric input and invalid ranges are checked before processing.
Resource Efficiency	Lightweight local JSON storage; no heavy external database required.
Testability	pytest tests cover data loading, grade calculation, and attendance validation.

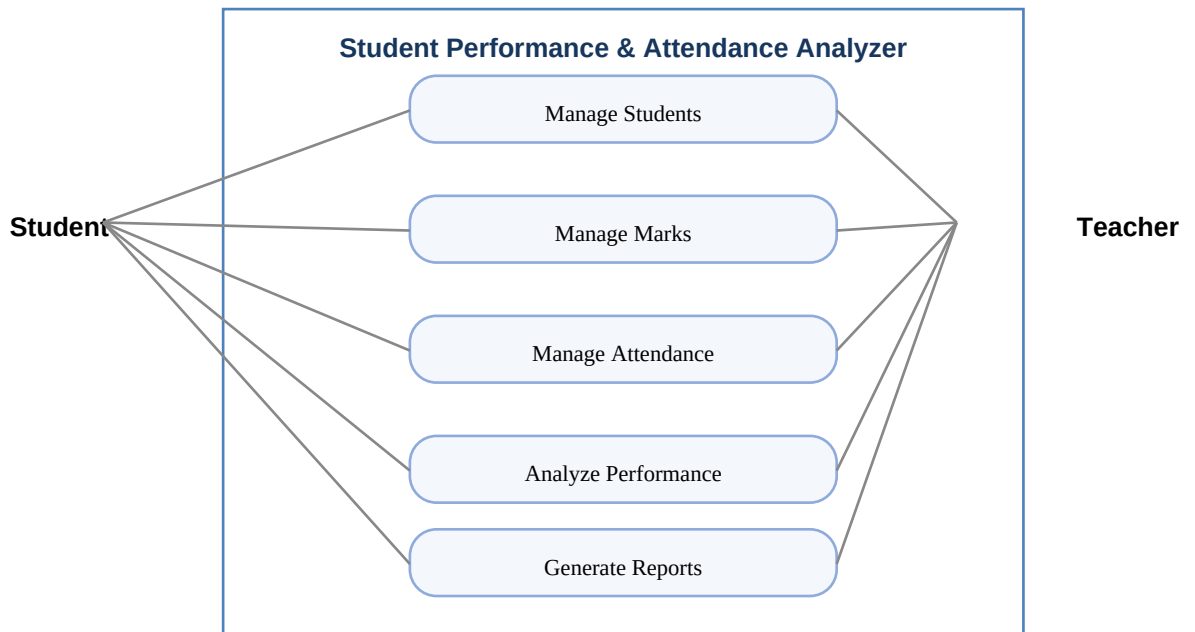
5. System Architecture



The architecture is modular. **main.py** acts as the controller, while specialized modules handle students, marks, attendance, analysis and reporting. The JSON file provides persistent local storage. The validator module is shared by data-entry modules.

6. Design Diagrams

6.1 Use Case Diagram



The use cases show the two primary actors and the main operations they can perform through the application.

6.2 Workflow Diagram

Start	↓
Open main menu	↓
Choose module	↓
Enter / retrieve student data	↓
Validate input	↓
Calculate / analyze	↓
Display or generate report	↓
Return to menu / exit	

6.3 Sequence Diagram

User	main.py	Module	students.json
Select operation	Route request	Process / validate	Read or write
Provide input	Pass input	Calculate / update	Persist data
Request result	Receive result	Return result	Return stored data
View output	Display output	—	—

6.4 Class / Component View

Component	Responsibility
main.py	Menu/controller and module integration
student.py	Student CRUD and JSON loading/saving
marks.py	Marks entry, percentage and grade calculation
attendance.py	Attendance entry and percentage calculation
analyzer.py	Combined performance analysis
reports.py	Consolidated report generation
validators.py	Input validation rules

6.5 Storage Design

7. Design Decisions & Rationale

- **Python:** Selected because the course focuses on Python programming and the project directly demonstrates Python concepts.
- **JSON storage:** Chosen as a simple, human-readable local persistence mechanism appropriate for a small academic project.
- **Modular files:** Separate modules reduce responsibility overlap and make the code easier to maintain and test.
- **Command-line interface:** Keeps the project focused on programming logic, validation, file handling and data processing.
- **pytest:** Provides repeatable automated tests for important calculation and validation logic.
- **Validation:** Prevents invalid marks and attendance values from being stored and handles non-numeric input without crashing.

8. Implementation Details

Project Structure

```
student-performance-analyzer/  
  main.py  
  student.py  
  marks.py  
  attendance.py  
  analyzer.py  
  reports.py  
  validators.py  
  requirements.txt  
  README.md  
  statement.md  
  data/students.json  
  tests/test_student.py  
  tests/test_marks.py  
  tests/test_attendance.py  
  docs/ (architecture, workflow, use case)
```

Core Calculations

```
Percentage = (Total Marks / Number of Subjects)  
  
Attendance Percentage = (Attended Classes / Total Classes) × 100
```

Grade boundaries implemented in the project are: A+ for 90 and above, A for 80–89.99, B for 70–79.99, C for 60–69.99, D for 50–59.99, and F below 50.

Performance Analysis

The analyzer combines academic percentage and attendance percentage. A student with both values at least 75% is classified as **Good**; values at least 60% but below the first condition are classified as **Average**; otherwise the result is **Needs Improvement**.

9. Screenshots / Results

The implemented application was tested through the command-line interface. The following results document representative outputs from the working project. Actual terminal screenshots can be added to this section before final submission if required.

Test / Operation	Observed Result
Add student	Student record saved successfully to students.json.

Test / Operation	Observed Result
Enter marks	Total, percentage and grade calculated and stored.
Enter attendance	Attendance percentage calculated and stored.
Performance analysis	Academic and attendance values combined into an overall category.
Generate report	Stored student information displayed in consolidated format.
Invalid marks: 105	Rejected because marks must be between 0 and 100.
Invalid attendance: 25/20	Rejected because attended classes cannot exceed total classes.
Non-numeric input: abc	Handled with an invalid-input message without program crash.

Example result: For 18 attended classes out of 20, the application calculates an attendance percentage of 90%.

10. Testing Approach

Testing combines input-validation checks performed during application use with automated unit tests written using pytest.

Automated tests included:

- Student data loading returns a list.
- Student records contain required fields when records exist.
- 95% produces grade A+.
- 85% produces grade A.
- 40% produces grade F.
- 20 total / 18 attended is valid.
- 20 total / 25 attended is invalid.
- Zero total classes is rejected.

Recorded test result: 8 automated tests passed.

Command used:

```
pytest
```

11. Challenges Faced

- Designing the project structure so each feature had a clear responsibility.
- Connecting separate Python modules through main.py.
- Handling invalid user input without terminating the application.
- Keeping JSON data consistent when marks and attendance are updated.
- Setting up pytest imports correctly for modules located in the project root.
- Preparing the repository structure and documentation required for GitHub submission.

12. Learnings & Key Takeaways

- Practical use of Python functions, loops, conditions and dictionaries.

- Reading and writing structured data using JSON.
- Creating modular programs using multiple Python files.
- Implementing input validation and exception handling.
- Writing automated tests with pytest.
- Designing basic software architecture and workflow documentation.
- Using Git for version control and preparing a GitHub repository.
- Writing README and project statement documentation.

13. Future Enhancements

- Add a graphical user interface.
- Support more subjects and configurable grading schemes.
- Export reports to PDF or CSV.
- Replace JSON storage with a relational database such as SQLite.
- Add charts for marks and attendance trends.
- Add authentication and role-based access for different users.
- Add search and filtering across generated reports.

14. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
- Python Documentation – Python language and standard library reference.
- pytest Documentation – Python testing framework documentation.
- Git Documentation – Distributed version control system documentation.

15. Project Submission Checklist

The following checklist maps the completed project to the requested VITyarthi submission artefacts.

Item	Status
Problem statement and objectives	Prepared in statement.md and this report
At least three functional modules	Implemented: Student, Marks, Attendance, Analysis, Reports
At least four non-functional requirements	Documented in this report
Architecture / workflow / UML-style diagrams	Prepared in docs/ and this report
Modular source code	Implemented across multiple Python files
Validation and error handling	Implemented
Automated testing	pytest tests included; 8 tests passed
README.md	Prepared
statement.md	Prepared
requirements.txt	Prepared

Item	Status
Git version control	Repository setup in progress
Public GitHub repository	To be completed before portal submission
Final screenshots	Recommended to add before final submission

Note: The VITyarthi guidelines require a detailed PDF report containing the listed sections and design artefacts, and require the GitHub repository to contain README.md, statement.md and organized project files. This report is structured to match those requirements.